Product Finder AI Agent Assignment

This assignment tests your full-stack AI development capabilities, framework adaptability, containerization/cloud engineering skills, and your understanding of production LLM evaluation methodologies.

You will build a natural language Product Finder AI Agent using Google's Agent Development Kit (ADK), deploy it to Google Cloud Platform (GCP) Cloud Run, build a lightweight frontend, and engineer an automated evaluation pipeline using DeepEval.

Part 1: Core Agent & Application Architecture

1. Backend Service (Google ADK)

- **Agent Logic:** Build a backend service using Google's Agent Development Kit (ADK) that processes natural language user queries and filters products from a provided mock dataset.
- **Deterministic Parameter Extraction:** Extracted parameters (e.g., category: clothing, max_price: 50.00) must directly trigger deterministic python/SQL filtering logic against the dataset. Do not use vector embeddings, RAG, or multi-agent orchestration for this task.
- **Example Scenarios:**
    - Category Search: "Show me all clothing items."
        - Filters dataset by category == "clothing".
    - Combined Search: "What clothing items are under $50?"
        - Filters dataset by category == "clothing" AND price <= 50.00 (returns matching items like a $45 Hoodie and $15 T-Shirt; excludes an $89 Jacket).

2. Containerization & Deployment

- **Docker:** Package your backend service into a clean container image (Dockerfile).
- **GCP Cloud Run:** Host your containerized service on Google Cloud Platform using Cloud Run as a serverless, public HTTPS endpoint.

Part 2: Agent Control Interface

To demonstrate production frontend engineering standards, your custom frontend must implement an **Agentic Control Interface** rather than a simple chat box. You are required to build and integrate four advanced UI components to manage asynchronous states, expose AI execution transparency, and synchronize interactive client state with agent context.

1. Multi-Stage Live Status & Skeleton Loader

- **Objective:** Eliminate black-box waiting states by visually communicating Cloud Run lifecycle events (including container cold starts) and Google ADK agent execution steps in real time.
- **Functional Description:**
    - Render an animated skeleton card grid while data is fetching, matching the dimensions and layout of the final product grid.
    - Render a dynamic status badge or progress indicator that cycles through granular execution stages as state events are received from the API or server-sent events.
- **Execution Stage Lifecycle:**
    - **Connecting / Warming:** Displayed when initiating the connection or waking up a cold serverless container instance.
    - **Parsing Intent:** Displayed while the agent analyzes the prompt and extracts search parameters.
    - **Executing Tool:** Displayed while the agent executes filtering logic against the product dataset.
    - **Rendering:** Displayed during final component hydration and layout painting.
- **Candidate Implementation Rules:**
    - Display a distinct, explicit user notification if backend execution time exceeds 1.5 seconds (e.g., "Waking up serverless container...").
    - Do not rely on generic circular loading spinners; use skeleton card layout placeholders.

2. Collapsible Tool Call Log (Agent Inspector)

- **Objective:** Expose the LLM's underlying function call execution and runtime metadata for inspectability, transparency, and debugging.
- **Functional Description:**
    - Position an accordion or collapsible control directly adjacent to or above the agent's text response.
    - When expanded, display details of the executed tool call, including the tool name, extracted parameters, request status, and server round-trip latency.
- **Required Metadata Fields:**
    - **Tool Name:** The explicit name of the function invoked by the agent (ex. filter_products).
    - **Extracted Parameters:** The structured keys and values parsed from the natural language prompt (e.g., category name, maximum price threshold, minimum price threshold).
    - **Latency:** Total execution and round-trip response time measured in milliseconds.
    - **Execution Status:** HTTP response code or success indicator.
- **Candidate Implementation Rules:**

    - The inspection panel must default to a collapsed state to avoid cluttering the primary user flow.
    - Display execution latency accurately based on measured API performance.
    - Formatted output must clearly distinguish keys and values without unformatted raw text dumps.

3. Active Constraint Chips Bar

- **Objective:** Provide an intuitive visual summary of all search constraints currently applied to the product dataset.
- **Functional Description:**
    - Render an interactive bar directly above the product grid containing individual dismissible badges ("chips") for every filter parameter extracted by the agent.
- **Required Component Properties:**
    - **Identifier:** Unique key representing the specific filter type (e.g., category, price ceiling, price floor).
    - **Display Label:** Clear, human-readable text summarizing the constraint (e.g., "Category: Clothing" or "Max Price: $50").
    - **Dismiss Action:** An interactive removal control (such as an "X" icon) on each badge.
- **Candidate Implementation Rules:**
    - Dynamically generate chips whenever the backend returns extracted tool parameters.
    - Provide a single-click "Clear All" action alongside individual chip controls when multiple filters are active.

4. Interactive State Mutation & Context Synchronization

- **Objective:** Enable bidirectional synchronization between manual UI filter controls and the ongoing conversation memory.
- **Functional Description:**
    - When a user dismisses a constraint chip or modifies a filter control, the frontend must immediately re-filter or re-query the product grid without requiring the user to type a new natural language prompt.
    - The updated constraint state must be synchronized back to the backend context so subsequent conversational turns respect the manually modified state.
- **Step-by-Step Interaction Workflow:**
    - **Initial Search:** The user submits a prompt ("Show me clothing under $50").
    - **State Render:** The system displays matching products along with active constraint chips ("Category: Clothing" and "Max Price: $50").
    - **User Mutation:** The user clicks to dismiss the "Max Price: $50" chip directly in the UI.
    - **Client Update:** The frontend instantly updates the product grid to show all clothing items regardless of price.

    - **Context Synchronization:** The frontend sends a state update payload to the agent endpoint reflecting the removal of the price constraint.
    - **Conversational Follow-Up:** The user asks a follow-up question ("Which ones are red?"). The agent evaluates the query against the mutated context (Category: Clothing, Price: Unrestricted).
- **Candidate Implementation Rules:**
    - Manual chip dismissal must immediately update the displayed product results.
    - State mutations must synchronize with the Cloud Run backend to ensure conversation memory remains accurate across multi-turn interactions.

Part 3: Automated Agent Evaluation (DeepEval)

To ensure high accuracy and resilience against edge cases, you will construct an automated evaluation pipeline that benchmarks your agent using the **DeepEval** testing framework.

1. Golden Dataset Construction (golden_dataset.json)

Construct a test dataset containing **at least 5 diverse prompts** paired with ground-truth target outputs. Your dataset must explicitly cover edge cases.

2. DeepEval Test Suite Implementation (eval_suite.py)

Implement a standalone python script or test suite utilizing the deepeval package to execute your evaluation dataset against your live agent API. You must implement at least 2 evaluation metrics:

1. **Deterministic Metric (Code/Exact Match):**
    - Use DeepEval's ExactMatchMetric (or a custom assertion metric) to check if your agent correctly extracted the target filtering parameters or exact product ID set from the query.
    - DeepEval Exact Match Documentation
2. **LLM-Based Evaluation Metric:**
    - Use an LLM-as-a-Judge metric such as G-Eval (GEval), AnswerRelevancyMetric, or FaithfulnessMetric to judge the natural language response quality, completeness, and adherence to constraints.
    - Define specific evaluation criteria for G-Eval (e.g., *"Assess whether the agent's summary accurately reflects the retrieved product prices and categories without inventing details"*).

    - DeepEval LLM Evals Documentation

Your initial agent should **not** achieve a 100% score. If it does, try making the tests more strict/detailed or add new metrics.

3. Iterative Optimization & Experimentation

- **Run Baseline Evaluation:** Execute your initial evaluation suite against your default ADK prompt/logic. Record the baseline pass rates and metric scores.
- **Test Prompt/System Improvements:** Conduct **at least 2 distinct iterations** to improve low-performing test cases (e.g., updating system instructions, adjusting tool call schemas, adding few-shot examples, or adding parameter fallback logic).
- **Document Results:** Track metric changes across iterations in a markdown file (EVALUATION.md). Detail your **best-performing change**, your **worst-performing change**, and key insights gained regarding LLM evaluation drift.

Live Technical Walkthrough

Be prepared during your technical interview to:

1. Demonstrate the live connection between your custom frontend and GCP Cloud Run backend.
2. Execute eval_suite.py and explain your choice of DeepEval metrics.
3. Discuss how you handled edge cases and system prompt iteration.

Technical Resources & Documentation

- Google Agent Development Kit (ADK): Google ADK Documentation
- GCP Cloud Run Quickstart: Deploying Python Services to Cloud Run
- DeepEval Documentation:
    - Getting Started with DeepEval
    - Exact Match Metric Reference
    - LLM-Based Metrics & G-Eval

Link for the Dataset: Dataset